

Recurrent Neural Networks and Transformers

Ref: Raschka, Sebastian; Liu, Yuxi (Hayden); Mirjalili, Vahid. Machine Learning with PyTorch and Scikit-Learn: Develop machine learning and deep learning models with Python.

Outline

What is an RNN and where can we use RNNs?

Understanding RNN and the mechanism

Implementing an RNN model using Pytorch

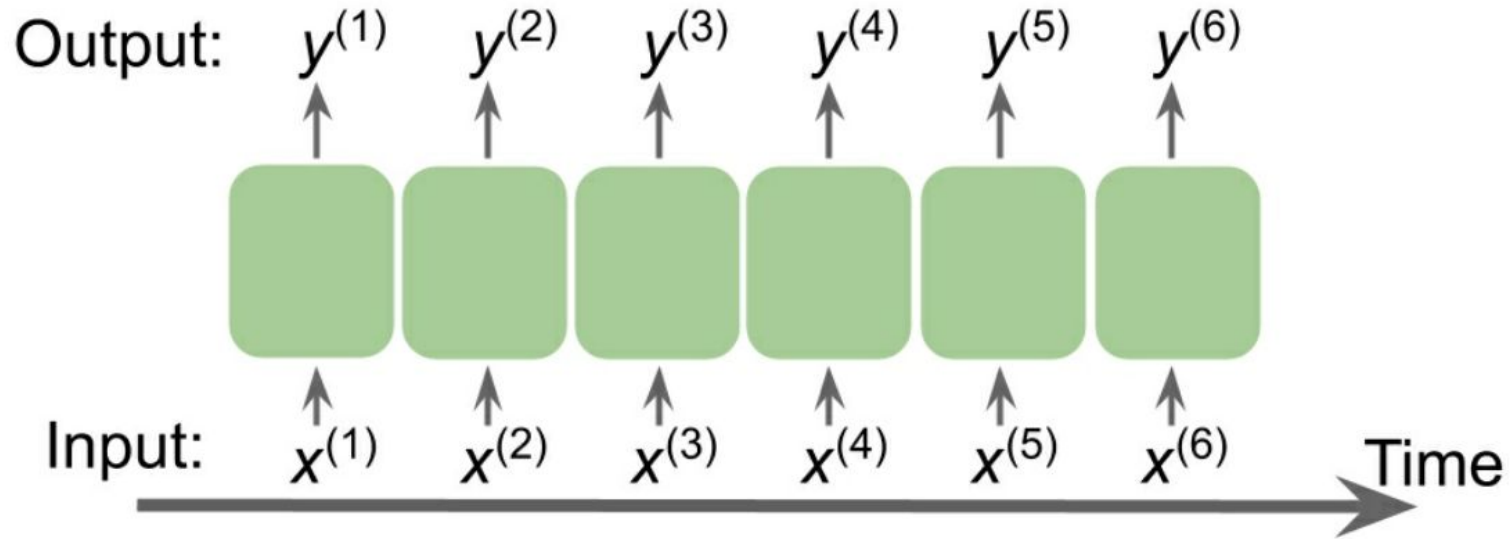
What is a transformer?

Understanding the original transformer network

Sequence data or sequences

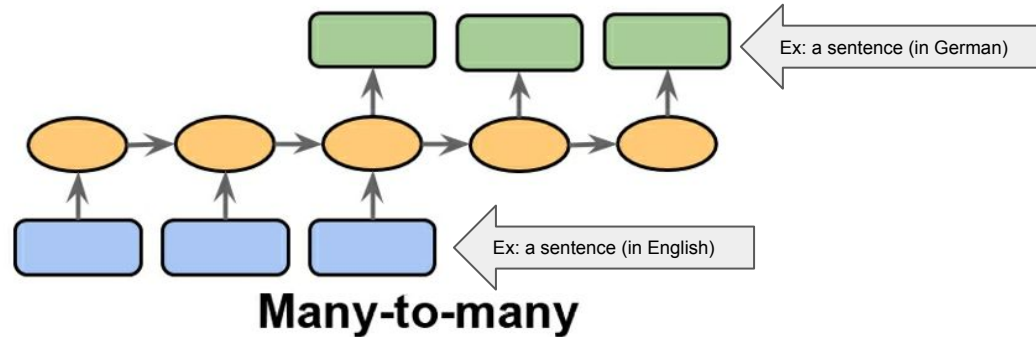
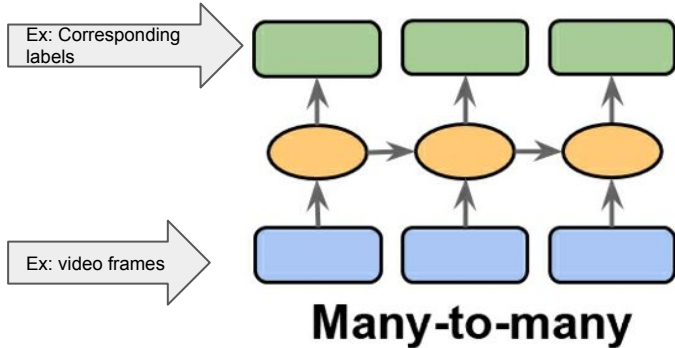
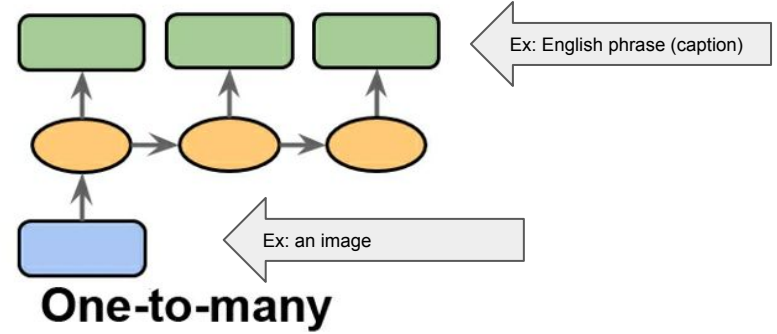
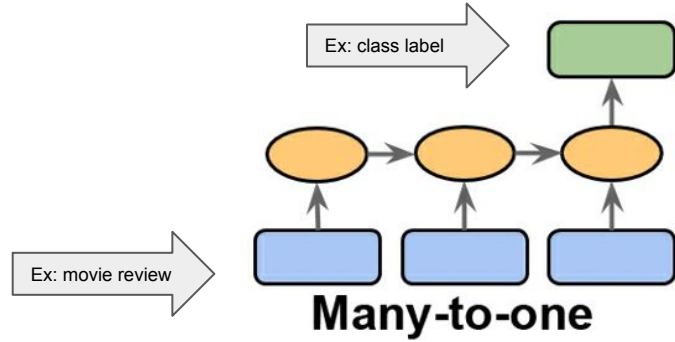
- In supervised learning:
 - We have used independent and identically distributed (IID) data
 - Meaning: training examples are mutually independent
- But, these are not true, when we deal with sequences, order matters
- Sequence vs time series
 - Time series data: is a special type of sequential data with a dimension for time.
 - Ex: stock prices, voice or speech records
 - Example of sequence data without time dimension: text, DNA sequence

Representation of sequences

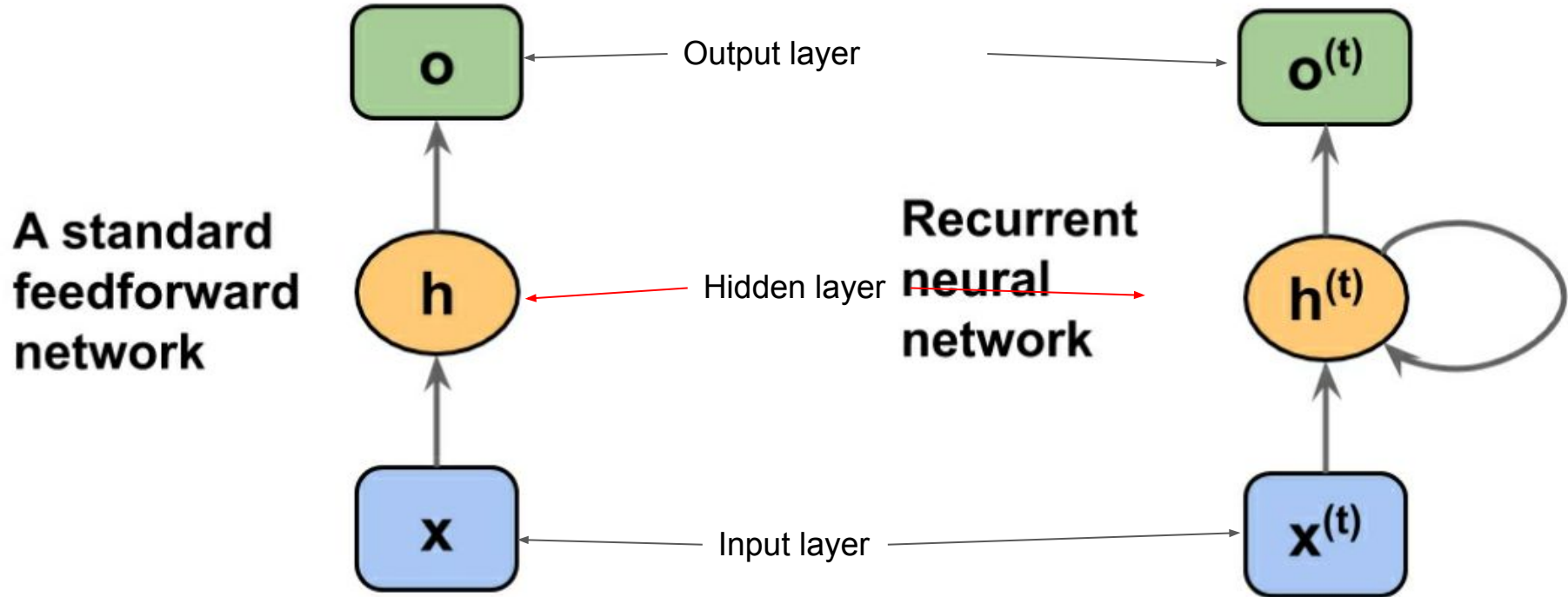


We have processed training data which are independent to each other for MLP and CNNs.

Different type of sequence tasks

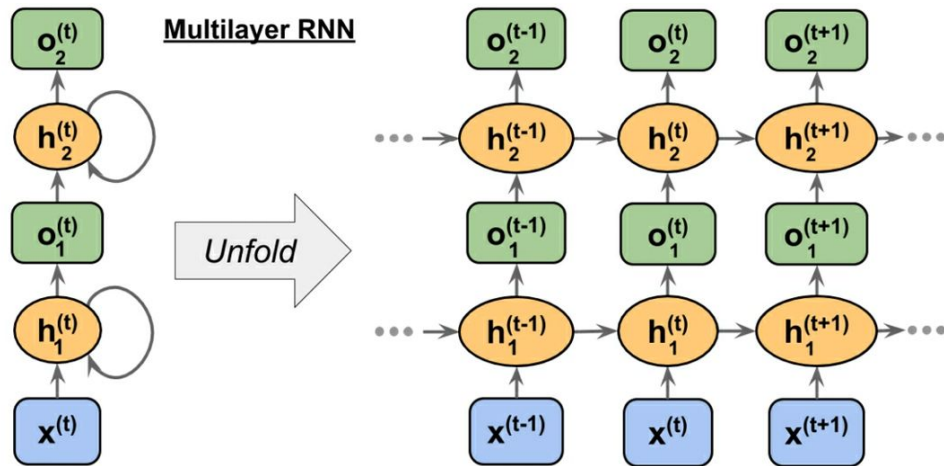
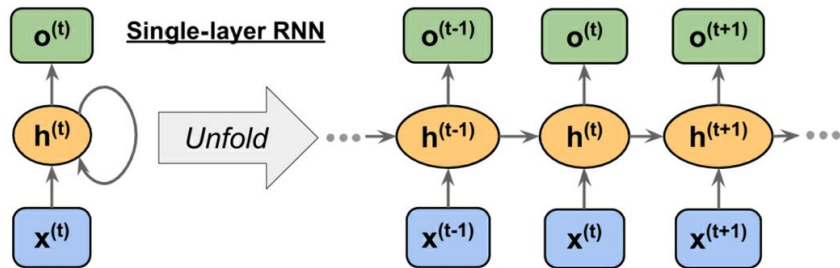


Recurrent Neural Network (RNN)



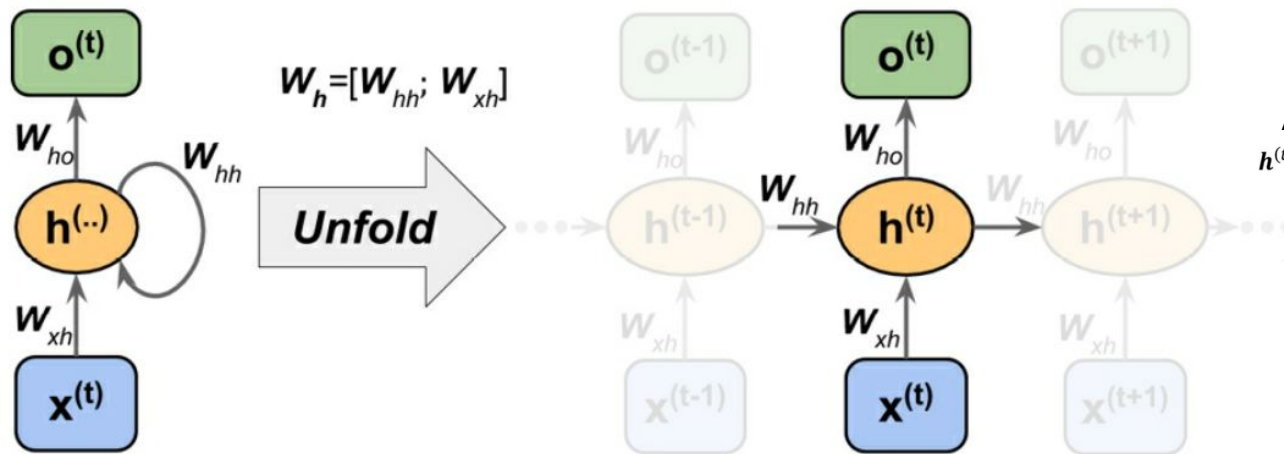
Note: x , h and o have many units.

RNNs



Activations in an RNN

- W_{xh} : The weight matrix between the input, $x^{(t)}$, and the hidden layer, h
- W_{hh} : The weight matrix associated with the recurrent edge
- W_{ho} : The weight matrix between the hidden layer and output layer



$$z_h^{(t)} = W_{xh}x^{(t)} + W_{hh}h^{(t-1)} + b_h$$

Activation of the hidden unit

$$h^{(t)} = \sigma_h(z_h^{(t)}) = \sigma_h(W_{xh}x^{(t)} + W_{hh}h^{(t-1)} + b_h)$$

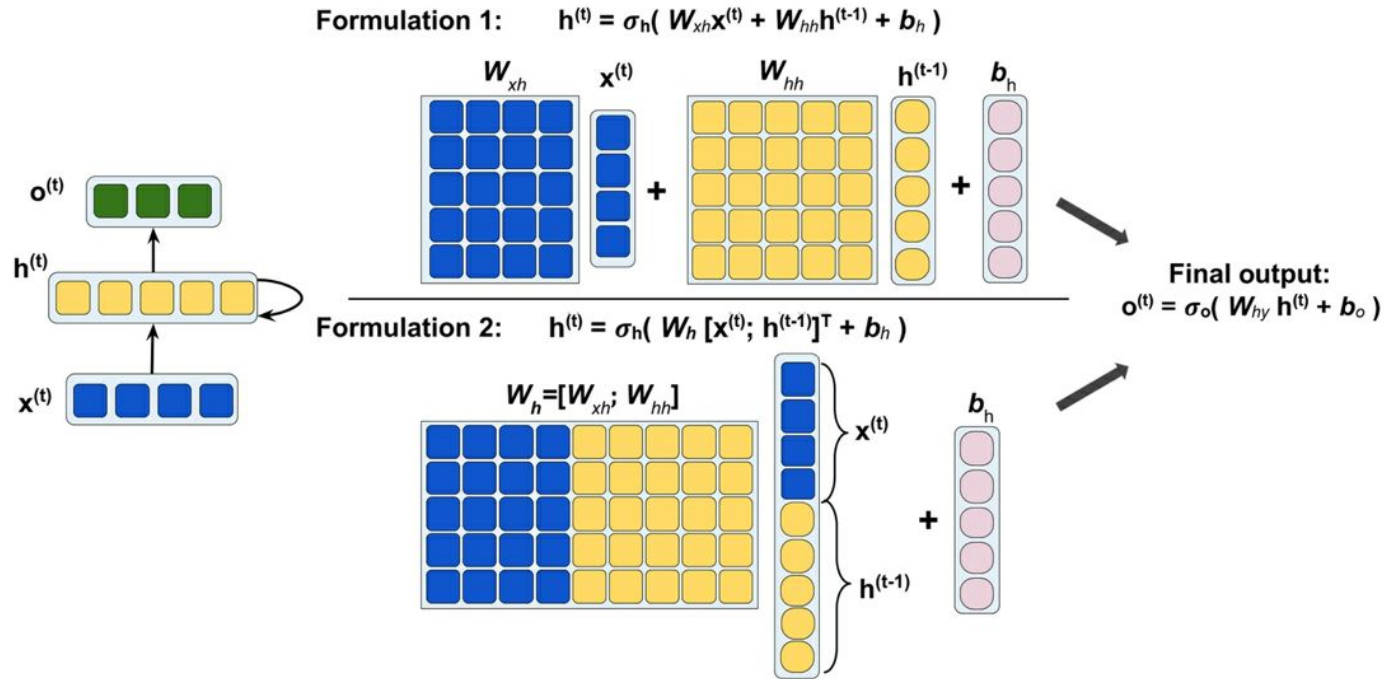
Another way of writing the same:

$$h^{(t)} = \sigma_h \left([W_{xh}; W_{hh}] \begin{bmatrix} x^{(t)} \\ h^{(t-1)} \end{bmatrix} + b_h \right)$$

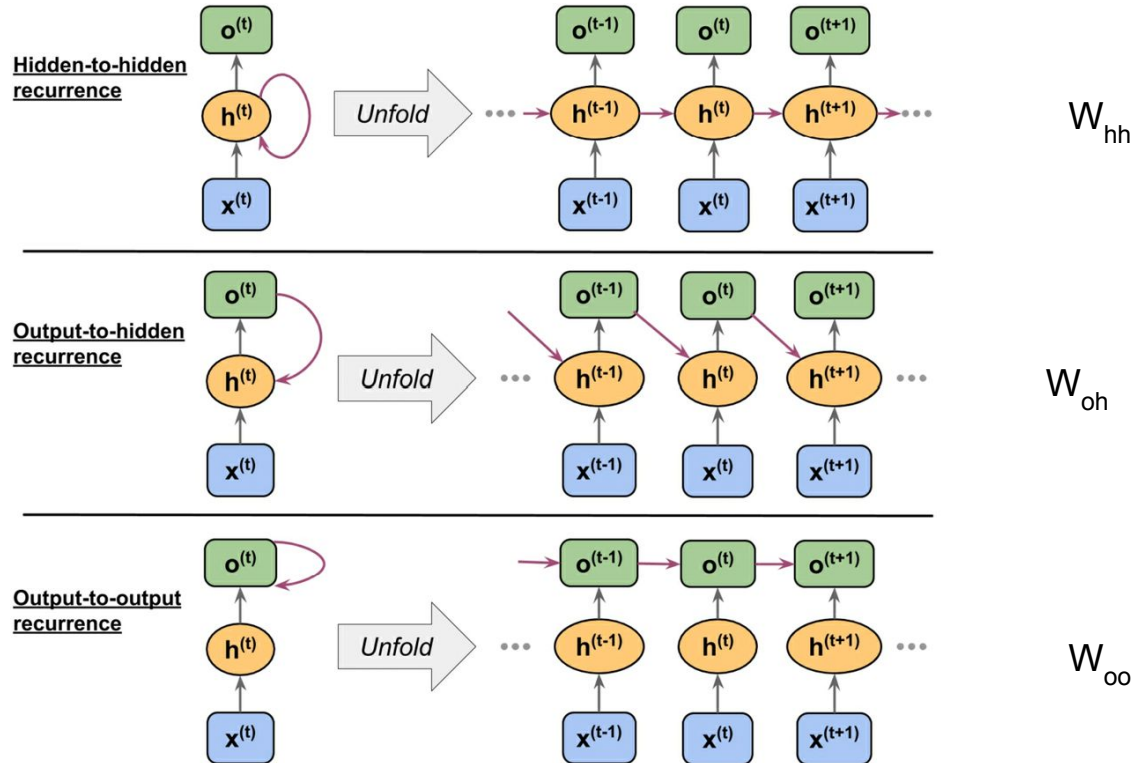
The activation of the output unit:

$$o^{(t)} = \sigma_o(W_{ho}h^{(t)} + b_o)$$

Activations in an RNN (big picture)



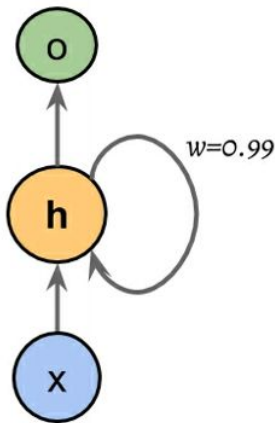
Different type of recurrent connections



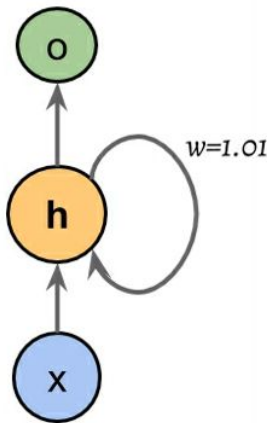
The challenges of learning long-range interactions

Backpropagation through time (BPTT) → vanishing and exploding gradient problems

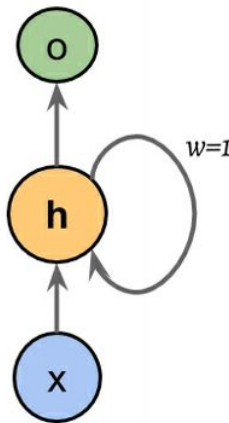
Vanishing gradient: $|w_{hh}| < 1$



Exploding gradient: $|w_{hh}| > 1$



Desirable: $|w_{hh}| = 1$

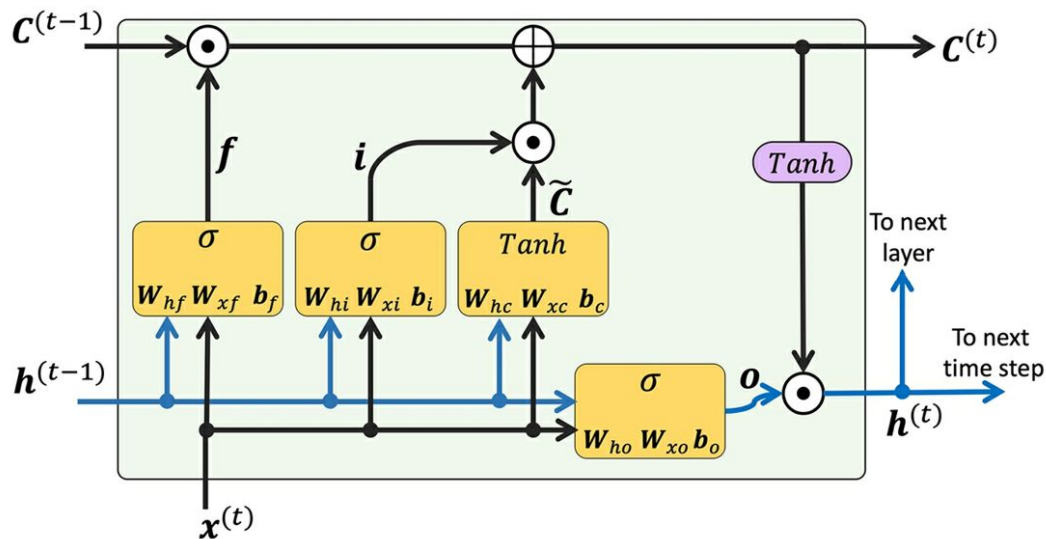


Three solutions:

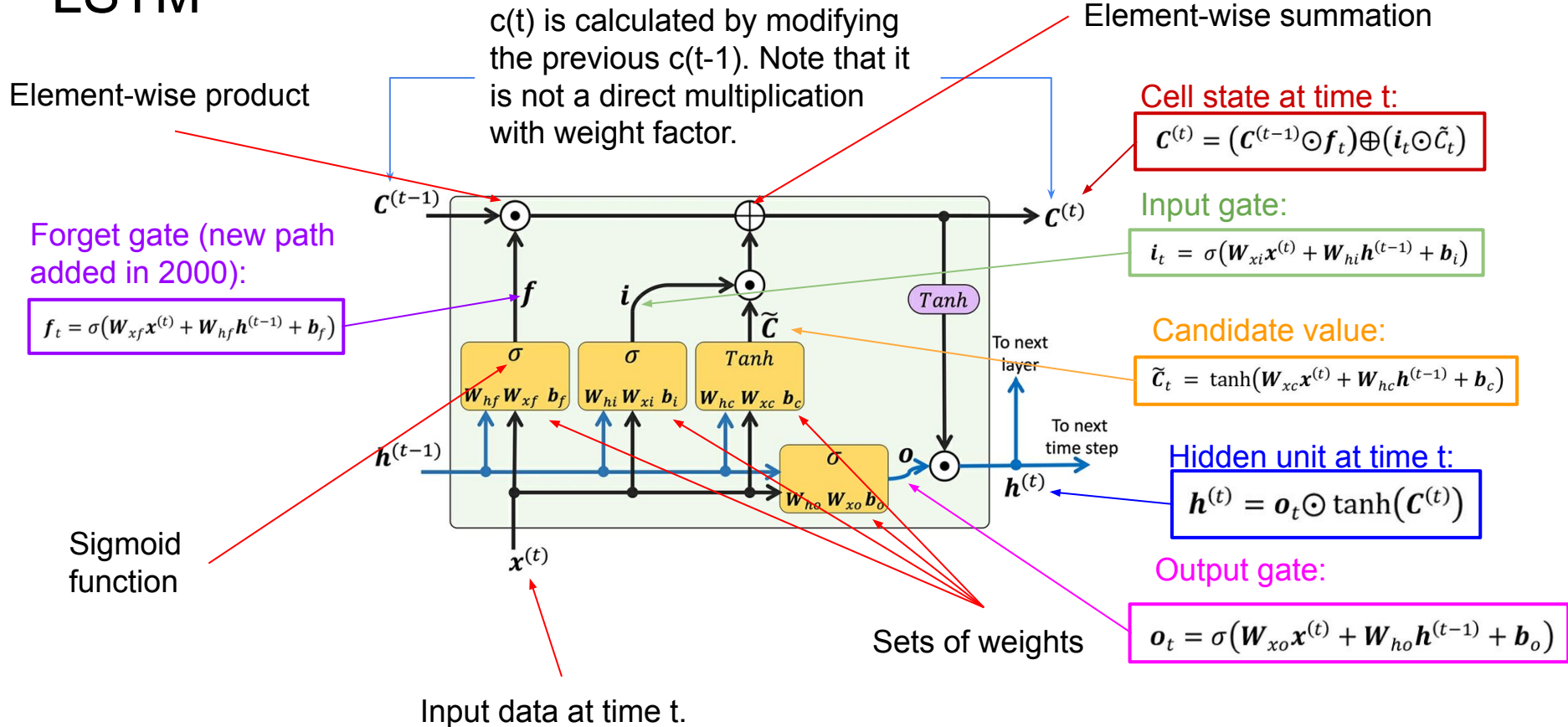
1. Gradient clipping: cut-off or threshold value for the gradient)
2. Truncated backpropagation through time (TBPTT): limits the number of time steps for backpropagation
3. LSTM

Long short-term memory (LSTM)

- Introduced in 1997 to overcome the vanishing gradient problem.
- The building block of an LSTM is a memory block
- Memory block $\rightarrow$ is used to replace the hidden layer of standard RNN



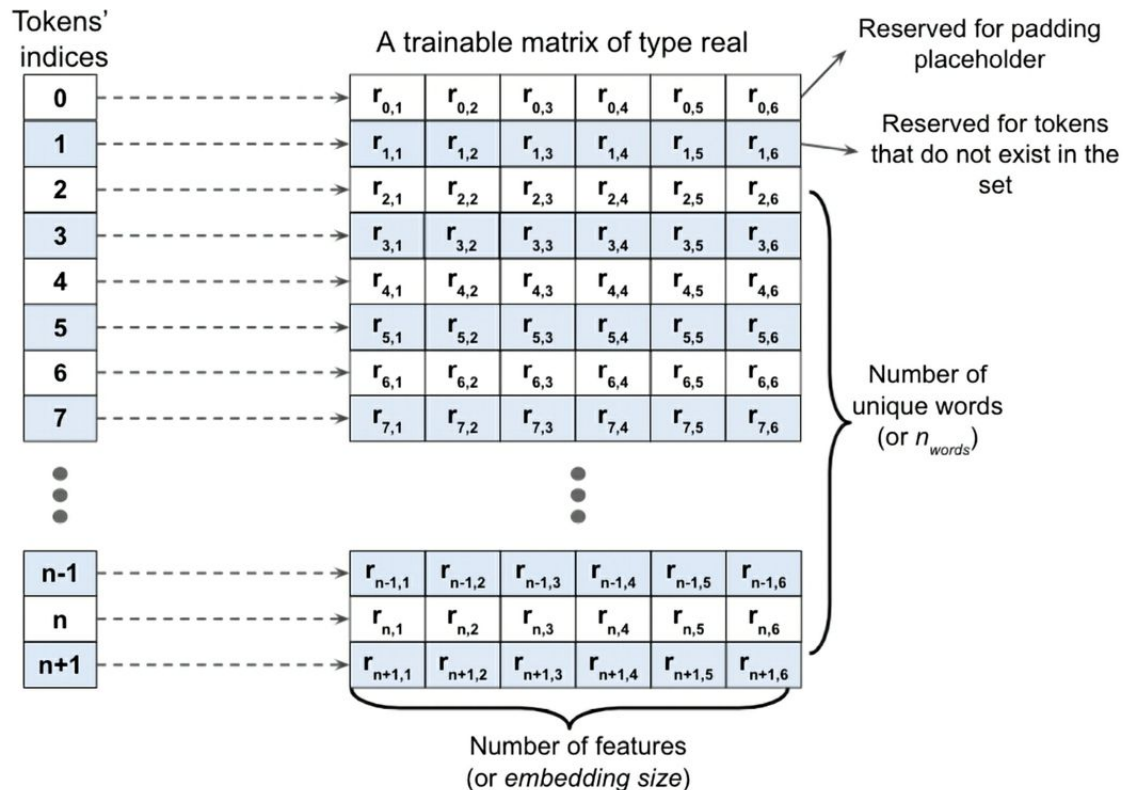
LSTM



Embedding layers for sentence encoding

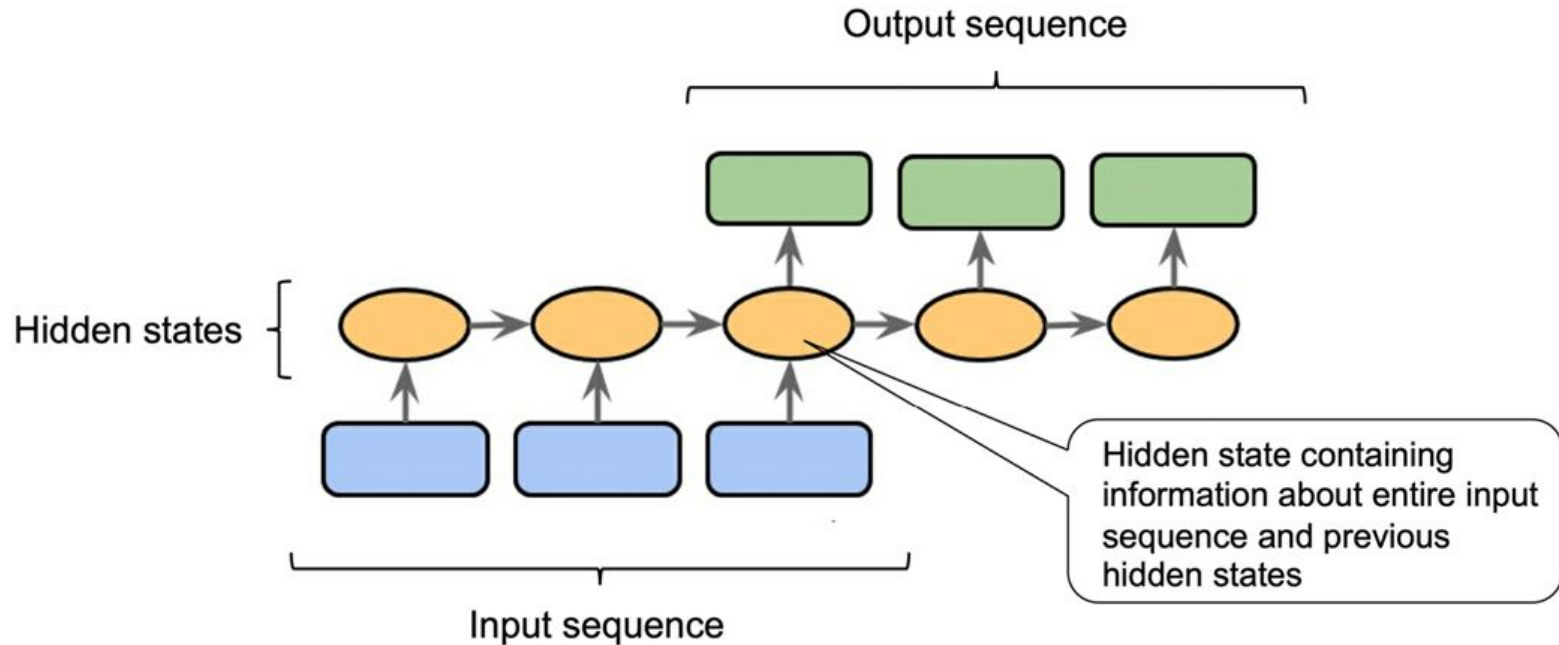
- What will happen if we use one-hot encoding?
- Embedding layer: is a feature-learning technique that we can utilize here to automatically learn the salient features to represent words in the dataset.
- For n_{words} we are going to select the size of embedding vector ($\text{embedding_dim} \ll n_{\text{words}}$) to represent the entire vocabulary.
- Advantages of embedding over one-hot encoding:
 - A reduction in the dimensionality of the feature space (decreasing the **curse of dimensionality**)
 - The extraction of salient features (embedding layers can be optimized or learned)

Embedding layers for sentence encoding



Transformers

Recall RNNs: Why is the RNN parsing the whole input sentence?



Recall RNNs: Why is the RNN parsing the whole input sentence?

German input sentence:

Kannst du mir helfen diesen Satz zu uebersetzen ?

Word-by-word translation (incorrect):

Can you me help this sentence to translate ?



The diagram shows a direct word-by-word mapping from German to English. Orange arrows point from each German word to its corresponding English word: 'Kannst' to 'Can', 'du' to 'you', 'mir' to 'me', 'helfen' to 'help', 'diesen' to 'this', 'Satz' to 'sentence', 'zu' to 'to', and 'uebersetzen' to 'translate'. This results in the incorrect sentence 'Can you me help this sentence to translate ?'.

German input sentence:

Kannst du mir helfen diesen Satz zu uebersetzen ?

Correct English translation:

Can you help me to translate this sentence ?



The diagram shows the correct semantic mapping from German to English. Orange arrows indicate the following connections: 'Kannst' to 'Can', 'du' to 'you', 'mir' to 'me', 'helfen' to 'help', 'diesen Satz' to 'this sentence', and 'zu uebersetzen' to 'to translate'. This results in the correct sentence 'Can you help me to translate this sentence ?'.

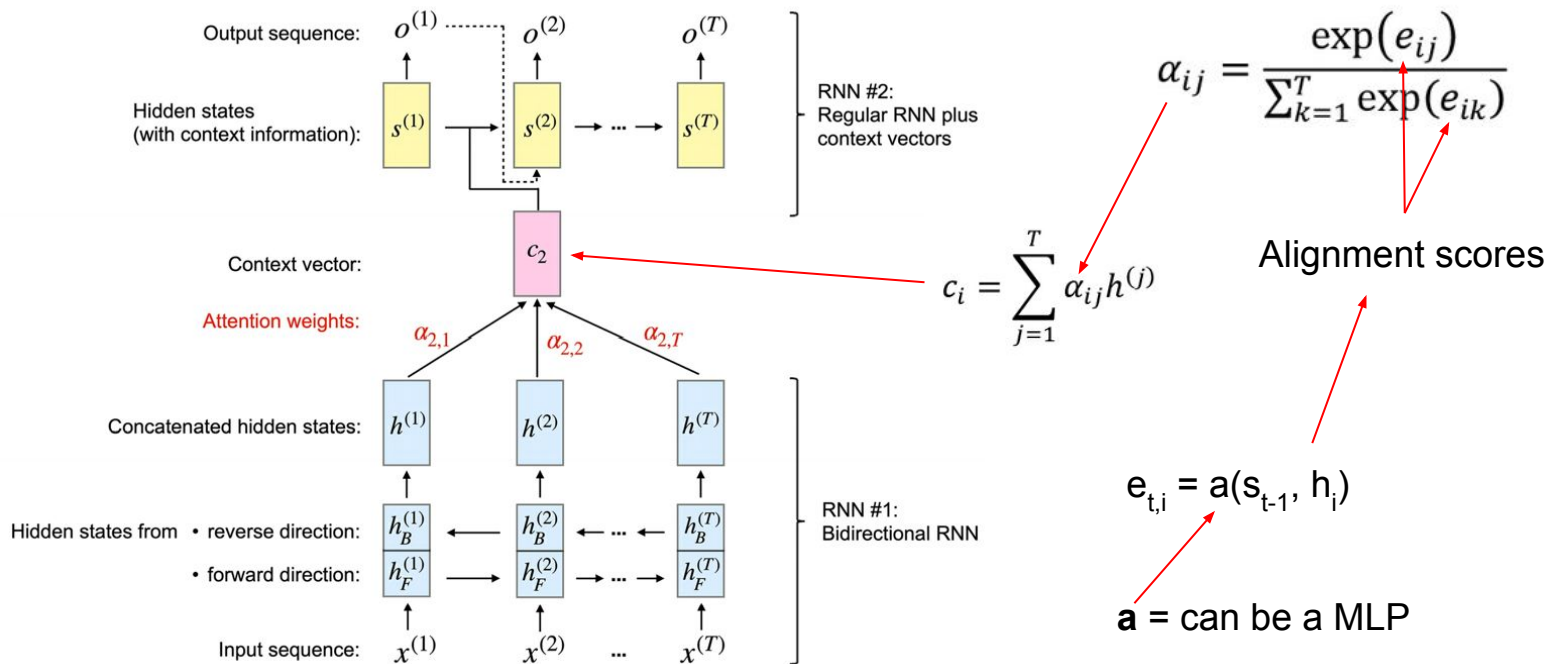
What are the limitations of RNNs?

- The RNN is trying to remember the entire input sequence via one single hidden unit before translating it.
- This means, it compress all the information into one hidden unit.

But in transformer network

- An attention mechanism access all input elements
- However, accessing all input sequence at every time step is overwhelming
- So, the attention mechanism assign different attention weights to each.

The original attention mechanism for RNNs

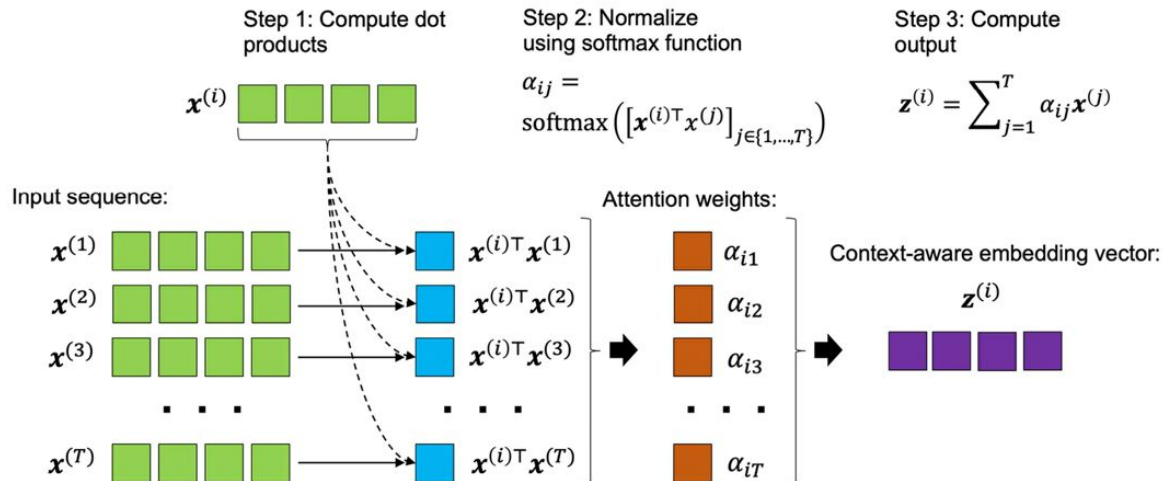


Self-attention (a basic form)

Ex:

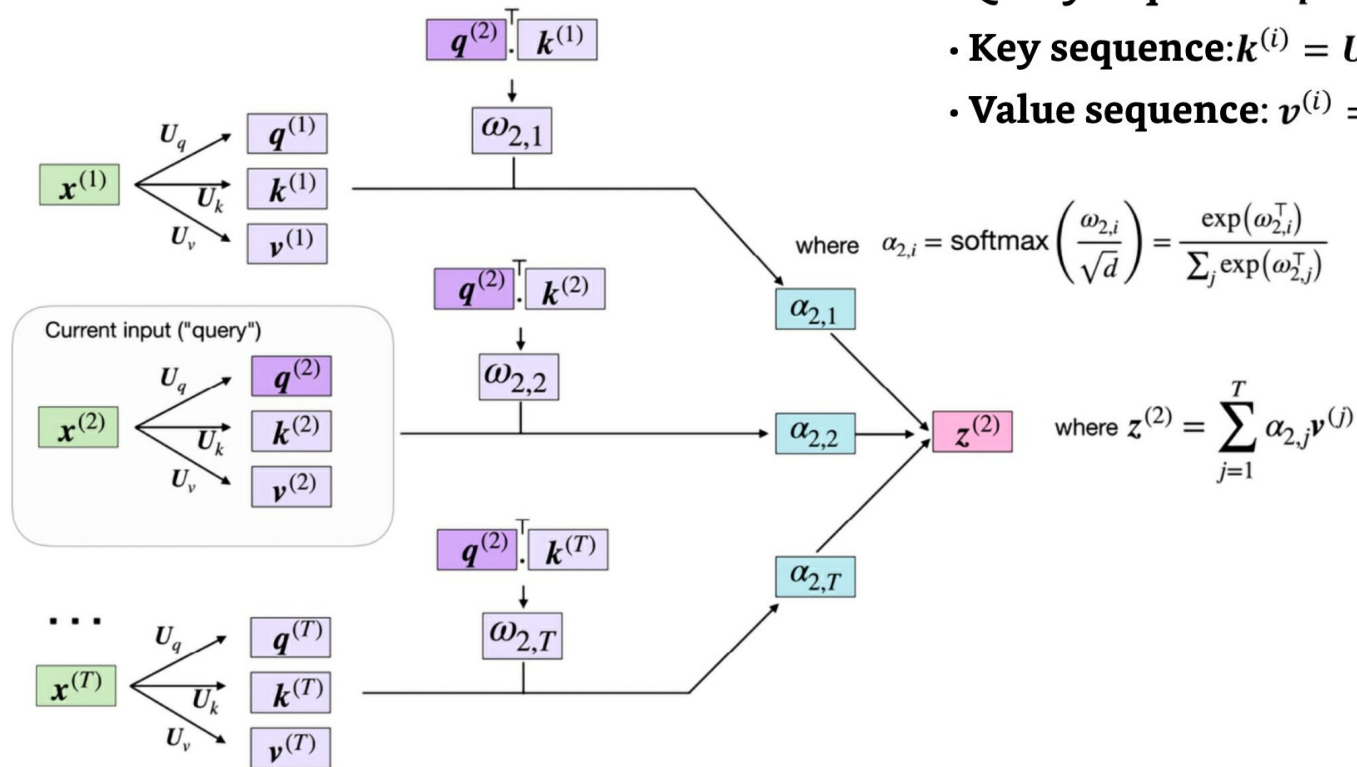
```
>>> import torch
>>> sentence = torch.tensor(
>>>     [0, # can
>>>       7, # you
>>>       1, # help
>>>       2, # me
>>>       5, # to
>>>       6, # translate
>>>       4, # this
>>>       3] # sentence
>>> )
>>> sentence
tensor([0, 7, 1, 2, 5, 6, 4, 3])
```

```
>>> torch.manual_seed(123)
>>> embed = torch.nn.Embedding(10, 16)
>>> embedded_sentence = embed(sentence).detach()
>>> embedded_sentence.shape
torch.Size([8, 16])
```

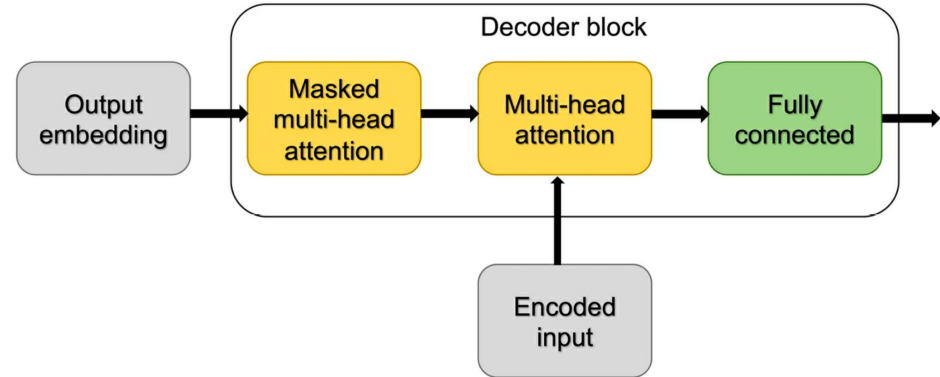
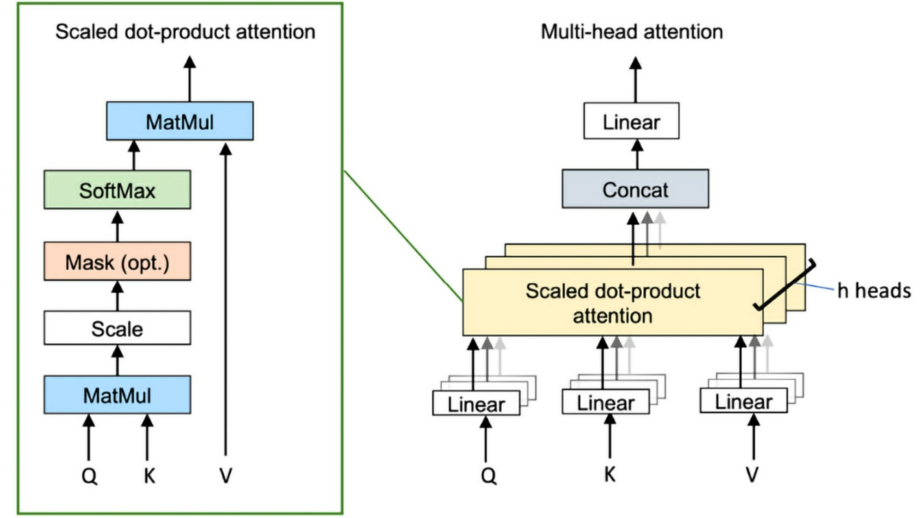
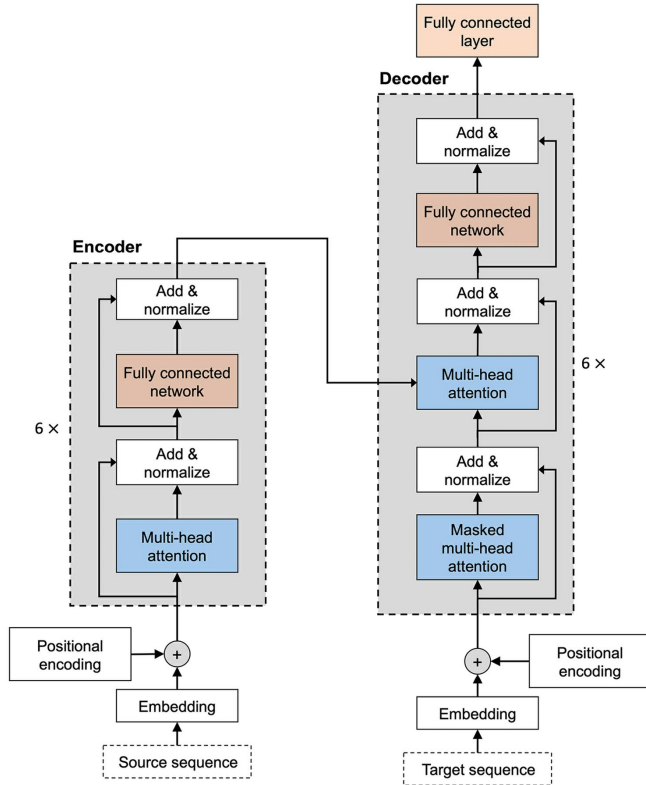


Parameterizing the self-attention mechanism: scaled dot-product attention

- **Query sequence:** $q^{(i)} = U_q x^{(i)}$ for $i \in [1, T]$
- **Key sequence:** $k^{(i)} = U_k x^{(i)}$ for $i \in [1, T]$
- **Value sequence:** $v^{(i)} = U_v x^{(i)}$ for $i \in [1, T]$



Original transformer network



<https://sebastianraschka.com/blog/2023/self-attention-from-scratch.html>

Mandatory assignment?